

Security Architecture - ELegislative

1) Purpose and Scope

This document describes the current security architecture of the ELegislative application, based on implemented backend and frontend code, and provides prioritized recommendations for hardening.

In scope:

- Identity and authentication
- Authorization and access control
- API and transport security
- Input validation and error handling
- File upload and media handling
- Realtime channel (Socket.IO)
- Data protection and operational controls

Out of scope:

- Cloud account posture (IAM, VPC, WAF) not represented in this repository
- CI/CD pipeline controls not represented in this repository

2) System Context and Trust Boundaries

Primary components:

- Web frontend (Vite/React) running in browser
- Node.js/Express backend API
- PostgreSQL database
- File storage under backend uploads directory, exposed via static route
- Socket.IO realtime server

Key trust boundaries:

- Browser to API boundary (cross-origin, token-based auth)
- API to database boundary (SQL access)
- API to static uploaded files boundary (public file serving)
- Socket.IO connection boundary (event-level trust)

3) Security Architecture Overview

3.1 Identity and Authentication

Implemented controls:

- Password hashing with bcrypt in auth controller.
- Access token (JWT) with 15-minute expiration.
- Refresh token (JWT) with 7-day expiration.
- Token verification middleware for protected routes.
- Auth endpoint rate limiting on register/login/refresh.

Implemented files:

- [backend/controllers/authController.js](#)
- [backend/middleware/auth.js](#)
- [backend/routes/auth.js](#)
- [frontend/src/api/api.js](#)

Frontend token behavior:

- Access and refresh tokens are stored in sessionStorage.
- Request interceptor automatically refreshes expired/missing access token via refresh endpoint.
- Authorization header uses Bearer token when available.

3.2 Authorization and Access Control

Implemented controls:

- Role-based middleware normalizes role name/id and enforces allowed roles.
- Route-level authorization on sensitive operations.
- Domain-specific authorization for committee meeting visibility and management.

Implemented files:

- [backend/middleware/roles.js](#)
- [backend/routes/committees.js](#)

Current committee meeting model:

- View rights include Admin, Vice Mayor, Secretary, committee chair, and committee members.

- Management rights are stricter and role-based (chair/committee secretary/admin/vice mayor path depending on route logic).

3.3 API Security and Transport

Implemented controls:

- CORS allowlist from environment variable CORS_ORIGINS.
- Explicit allowed methods and headers.
- trust proxy enabled for correct behavior behind Render reverse proxy.
- Global JSON body parsing and centralized error handler.

Implemented files:

- [backend/server.js](#)
- [backend/middleware/errorHandler.js](#)

3.4 Input Validation and Request Hygiene

Implemented controls:

- Joi schemas for key resources (auth, ordinances, resolutions, minutes, messaging).
- Validation middleware strips unknown body fields (except query behavior as designed).
- Validation failures converted to structured errors.

Implemented files:

- [backend/middleware/validation.js](#)
- [backend/validators/schemas.js](#)

3.5 Database and Data Access Security

Implemented controls:

- pg pool with parameterized queries used broadly in services/models.
- Database URL from environment variables.
- Render SSL mode enabled for render.com connection strings.

Implemented file:

- [backend/db.js](#)

3.6 File Upload and Media Security

Implemented controls:

- Multer storage to specific directories.
- Recording upload middleware enforces video MIME type and 1 GB limit.
- Upload filenames sanitized for recordings.

Implemented files:

- [backend/middleware/meetingRecordingUpload.js](#)
- [backend/middleware/sessionRecordingUpload.js](#)
- [backend/middleware/upload.js](#)

Important architectural note:

- uploads is served as a public static path from backend.
- Access to uploaded files currently depends on obscurity/path knowledge, not per-file authorization checks.

3.7 Realtime Security (Socket.IO)

Implemented behavior:

- Socket server configured with CORS allowlist.
- Clients can join role rooms and user rooms based on emitted events.

Implemented file:

- [backend/socket.js](#)

Security gap:

- No JWT validation in Socket.IO handshake.
- joinRole and joinUser events currently trust client-supplied role/user identifiers.
- This enables potential unauthorized room subscription if abused.

4) Security Data Flows

4.1 Login and Session Flow

1. User submits credentials to login endpoint.
2. Backend verifies password hash and issues access token and refresh token.
3. Frontend stores tokens in sessionStorage.
4. Frontend sends access token in Authorization header.
5. If access token is expired/missing, frontend calls refresh endpoint and retries.

4.2 Protected API Request Flow

1. Frontend interceptor attaches Bearer token.
2. Backend auth middleware verifies JWT and sets req.user.
3. Role/domain middleware enforces route policy.
4. Controller/service executes data operation.

4.3 Committee Meeting Access Flow

1. User loads measure or committee meetings page.
2. Backend committee route checks role/membership rules.
3. Frontend additionally applies UI-level visibility checks.
4. User can view and optionally join online meeting links when present.

5) Current Strengths

- Short-lived access tokens with refresh pattern.
- Password hashing via bcrypt.
- Route-level role middleware and domain-specific committee checks.
- Validation middleware with reusable Joi schemas.
- Centralized production-safe error response behavior.
- Auth endpoint rate limiting.
- Proxy trust configured for hosted environment compatibility.

6) Security Risks and Gaps

High

1. Socket.IO identity trust gap
 - Risk: Unauthorized users can claim rooms by sending arbitrary role/user values.
 - Impact: Confidential notification leakage and event spoofing exposure.
2. Public static uploads exposure
 - Risk: Uploaded files are publicly retrievable by path.
 - Impact: Potential unauthorized access to recordings/documents.

Medium

3. Token storage in sessionStorage

- Risk: Any successful XSS can exfiltrate tokens.
- Impact: Session hijacking window until expiration/revocation.

4. No refresh token revocation/rotation mechanism

- Risk: Stolen refresh tokens remain valid until expiry.
- Impact: Prolonged unauthorized session renewal.

5. Uneven rate limiting coverage

- Risk: Non-auth APIs can be brute-forced or abused.
- Impact: Abuse, scraping, and denial-of-service pressure.

Low to Medium

6. Generic upload middleware for ordinances has no MIME/size constraints.

- Risk: Oversized or unexpected file type uploads.
- Impact: Storage abuse and downstream handling risk.

7. CORS configuration relies on strict environment hygiene.

- Risk: Misconfigured origin list broadens trust unexpectedly.
- Impact: Increased cross-origin attack surface.

7) Recommended Target Security Architecture

7.1 Identity and Session Hardening

- Move refresh token to HttpOnly, Secure, SameSite cookie.
- Keep access token short-lived and in-memory where feasible.
- Implement refresh token rotation and revocation table.
- Add logout invalidation for refresh token family.

7.2 Realtime Channel Hardening

- Enforce JWT verification in Socket.IO handshake middleware.
- Derive user and role from verified token only.
- Remove or ignore client-provided role/user IDs for room joins.
- Authorize session-specific live stream events by role and participation.

7.3 Authorization Consistency

- Centralize policy decisions in backend services (ABAC/RBAC policy module).
- Keep frontend checks as UX only, never as security boundary.
- Add integration tests for critical role/membership combinations.

7.4 API Protection

- Add rate limits per route category beyond auth (read-heavy, write-heavy).
- Add request size limits and stricter body parser policies.
- Add security headers middleware (helmet) with CSP where compatible.

7.5 File and Recording Protection

- Store sensitive uploads outside public static path.
- Serve files through authorized download endpoints with token checks.
- Enforce extension and MIME allowlists everywhere.
- Add malware scanning and content-type verification for non-video uploads.

7.6 Monitoring and Incident Response

- Structured audit logs for auth failures, privilege denials, and upload actions.
- Alerting on anomalous refresh rates, repeated 401/403 spikes, and high upload frequency.
- Retention and review policy for audit/security logs.

8) Security Control Matrix (Current State)

- Authentication: Implemented
- Authorization (RBAC): Implemented with domain custom checks
- Input validation: Implemented on key endpoints
- Auth rate limiting: Implemented
- Global rate limiting: Partial
- Realtime authentication: Missing
- Token revocation: Missing
- Upload access control: Partial
- Security headers baseline: Not explicit
- Audit logging: Present in multiple domain flows

9) Verification Plan

Recommended tests for acceptance:

1. Role-based API tests for Admin, Vice Mayor, Secretary, Councilor, committee member.
2. Negative tests for unauthorized socket room joins.
3. Upload abuse tests (type/size/path traversal attempts).
4. Token lifecycle tests (expiry, refresh, logout, replay).
5. Access tests for recordings and uploaded files by unauthorized users.

10) Operational Checklist

Before production release:

- Verify JWT secret and refresh secret rotation policy.
- Restrict CORS_ORIGINS to exact production origins.
- Confirm HTTPS-only deployment and HSTS at edge.
- Enable DB backups and tested restore.
- Add runtime monitoring and alert thresholds.

11) Revision Notes

Generated from repository implementation state as of 2026-07-14. Primary references:

- [backend/server.js](#)
- [backend/controllers/authController.js](#)
- [backend/middleware/auth.js](#)
- [backend/middleware/roles.js](#)
- [backend/routes/auth.js](#)
- [backend/routes/committees.js](#)
- [backend/middleware/validation.js](#)
- [backend/validators/schemas.js](#)
- [backend/middleware/meetingRecordingUpload.js](#)
- [backend/middleware/sessionRecordingUpload.js](#)
- [backend/middleware/upload.js](#)
- [backend/socket.js](#)
- [backend/db.js](#)
- [frontend/src/api/api.js](#)